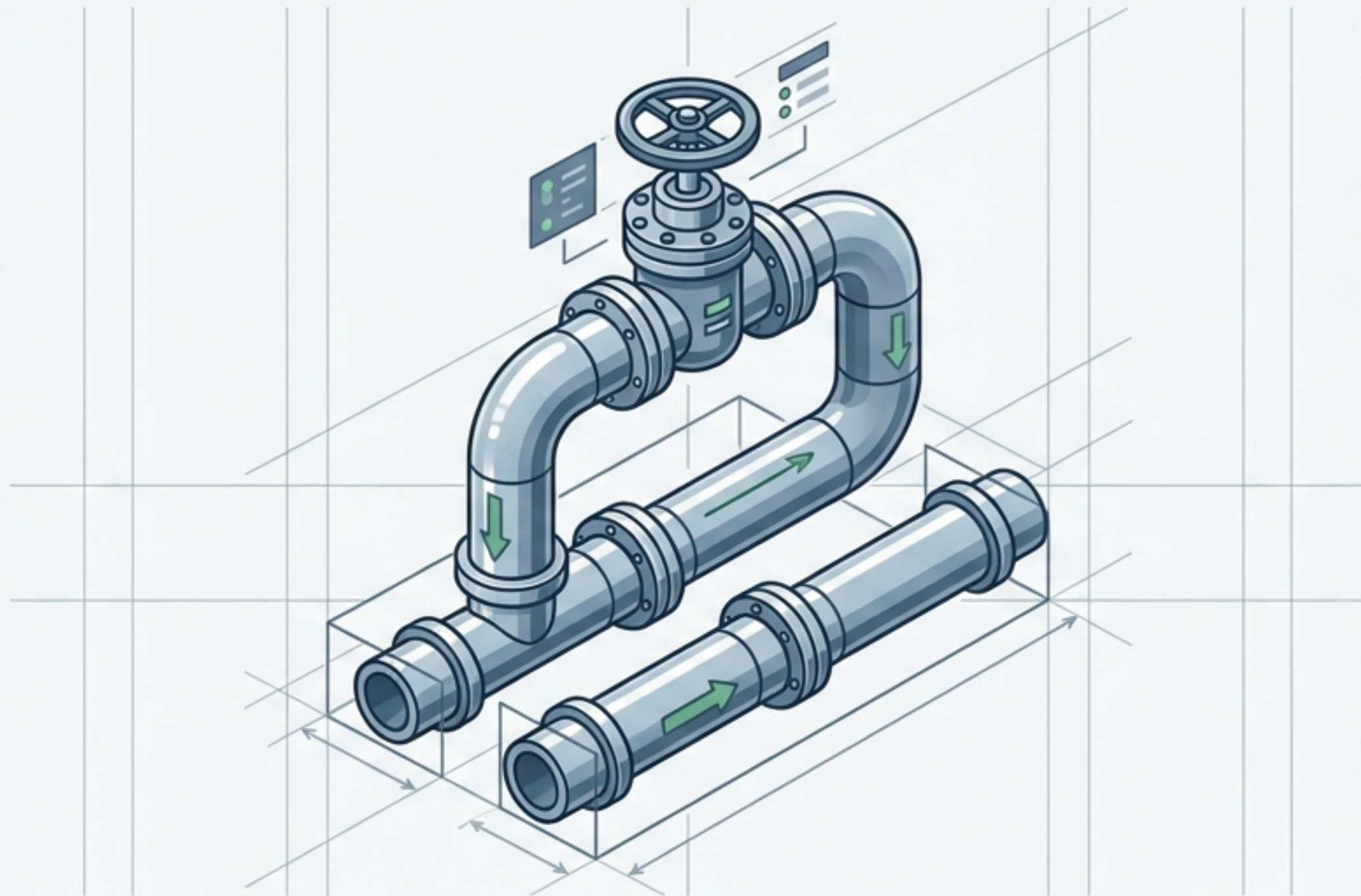
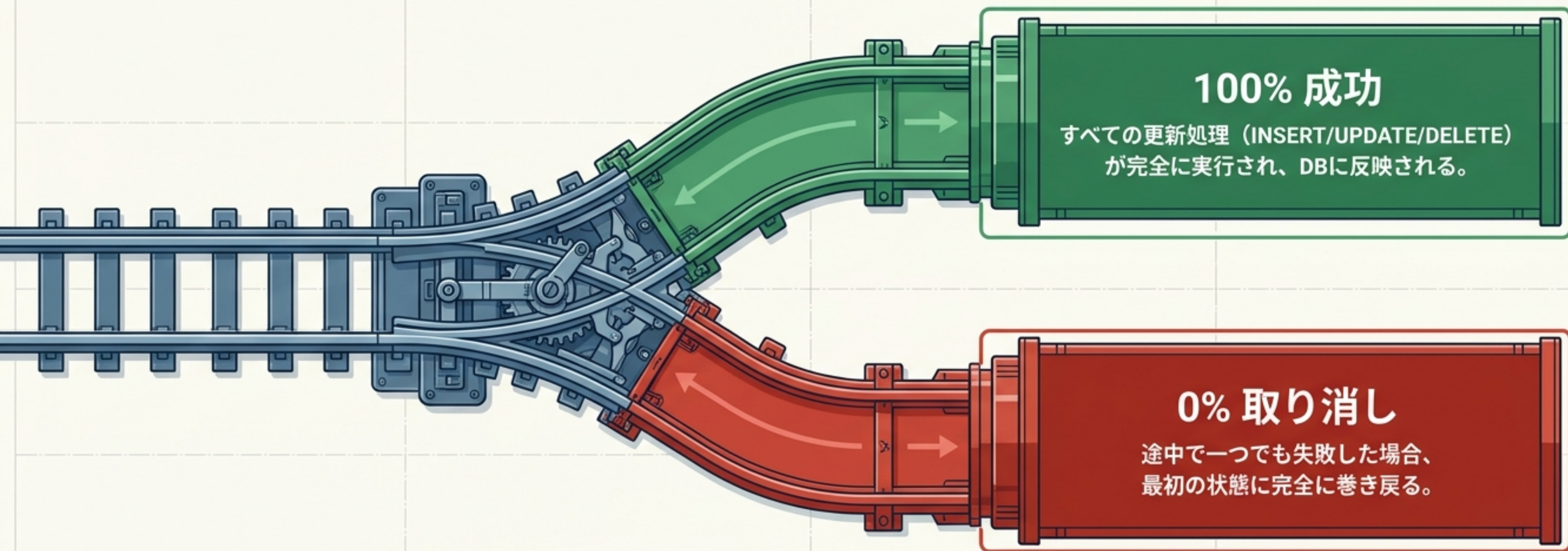




JDBCトランザクションの確実な制御

DAOパターンにおけるConnectionの絶対的ルールと内部構造

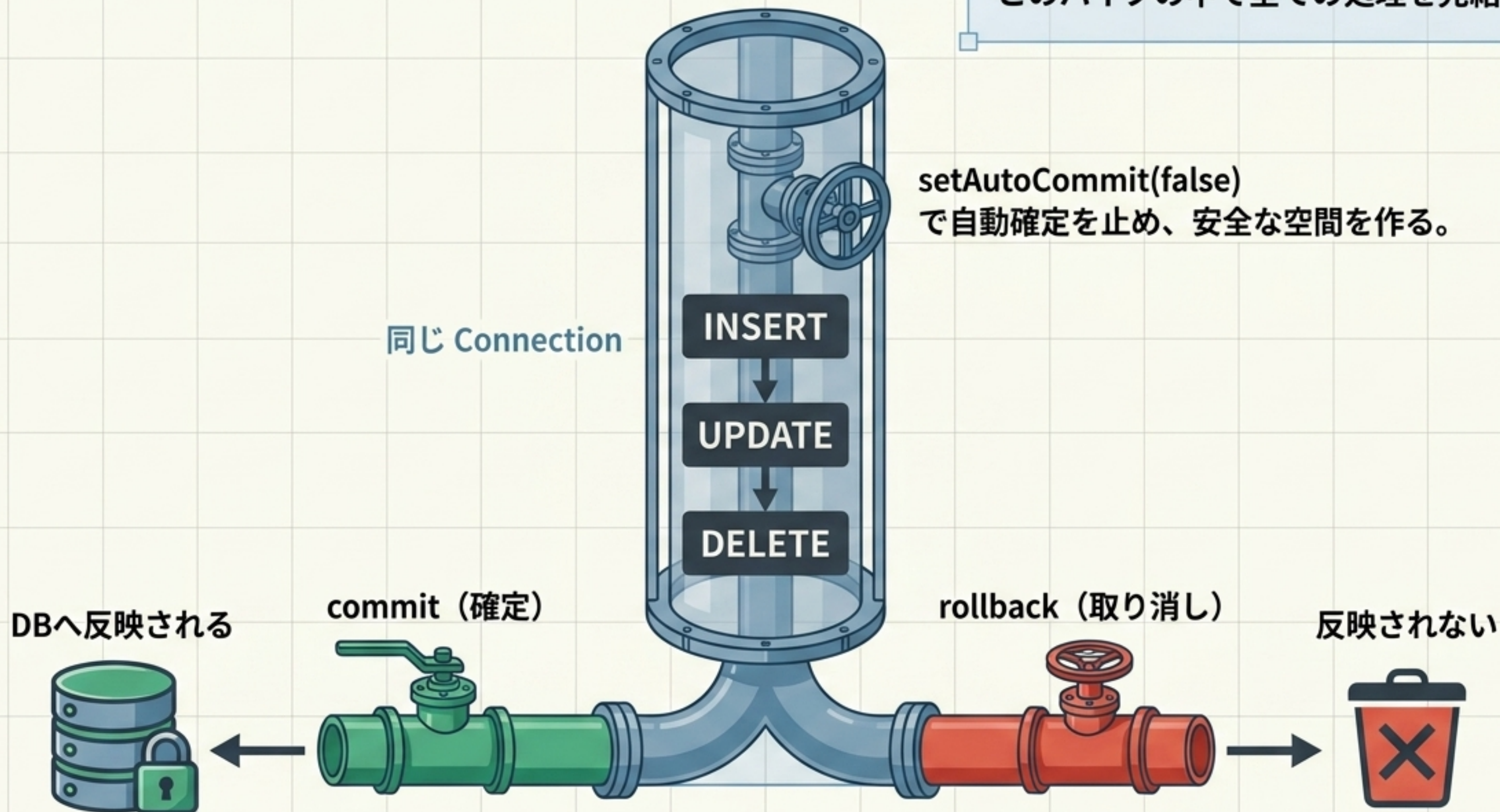
トランザクションの鉄則：「すべて成功する」か「すべて無かったことになる」か。



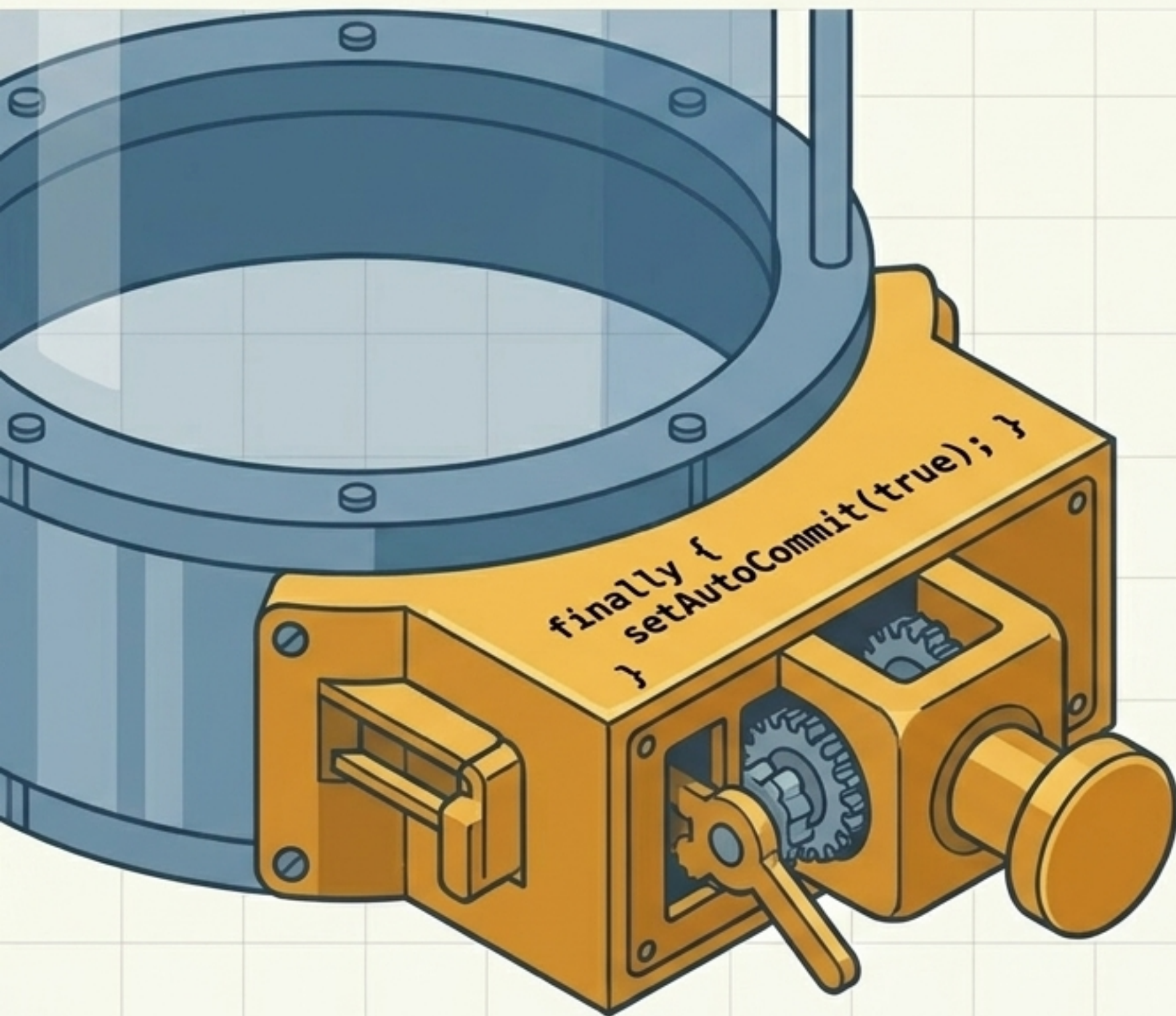
この「0か100か」の絶対的な制御を実現する唯一の鍵が、JDBCにおける Connection オブジェクトの適切な管理です。

トランザクションの黄金律 (Golden Path)

主役は常に「同じ Connection」。
このパイプの中で全ての処理を完結させる。



事故防止の最終防衛線：「finally」



Mechanism Map Diagram



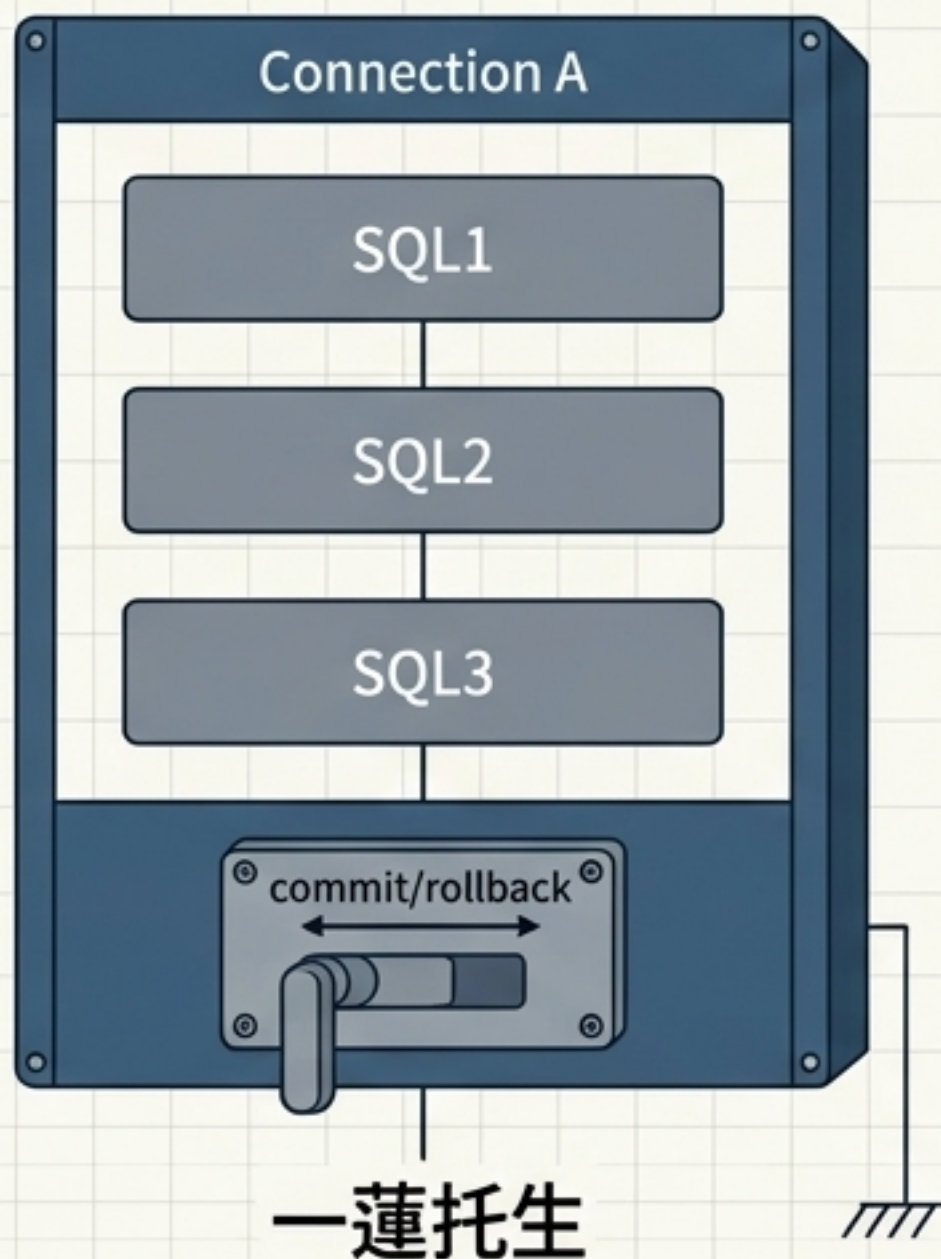
!

理由：コネクションプールに戻す前に「元の状態（true）」にリセットするため。

警告：これを忘れると、次にこのConnectionを受け取った別の処理が、意図せずランザクションに巻き込まれる大事故に繋がる。

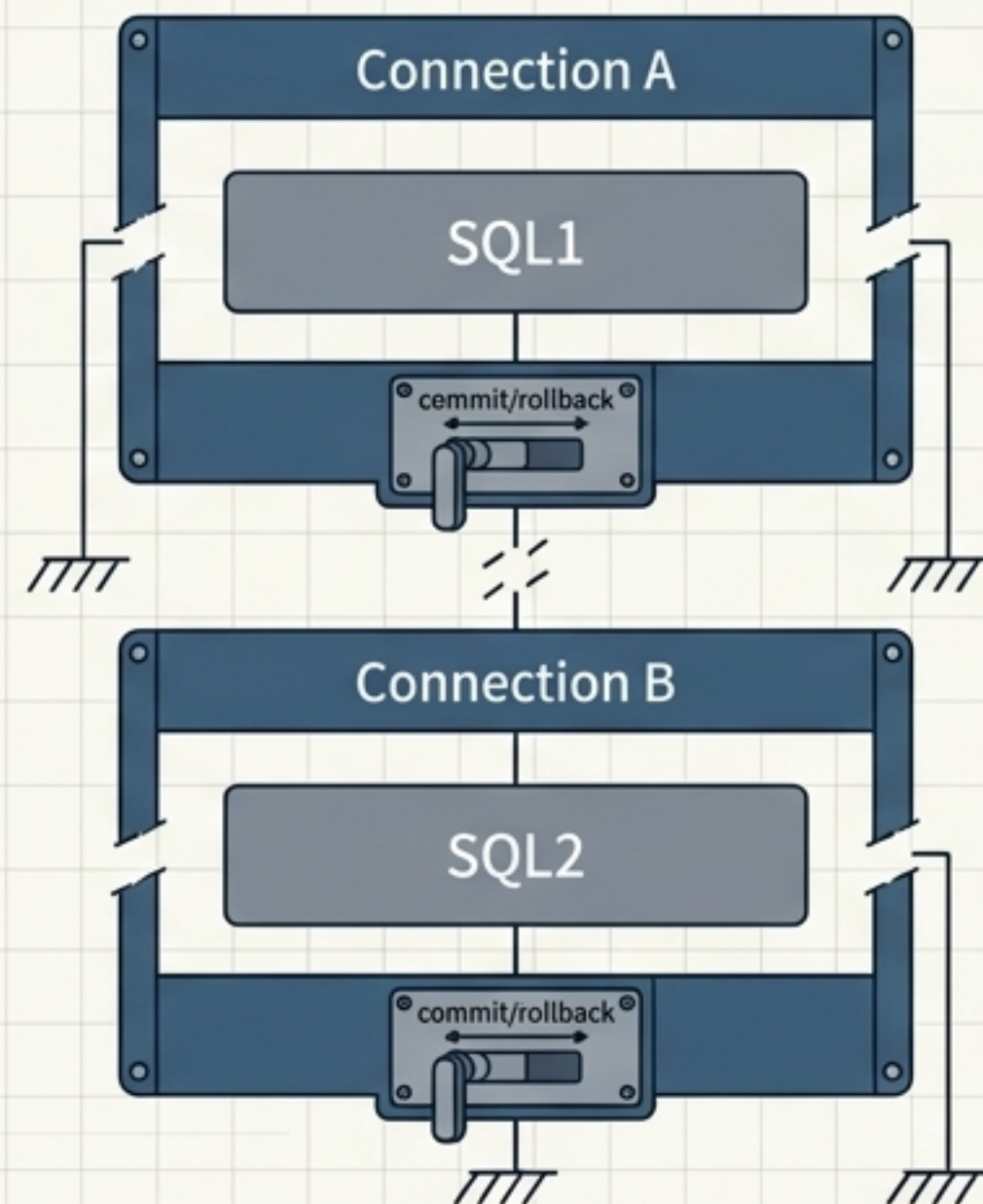
なぜ「同じConnection」の共有が絶対条件なのか？

✓ 良い例：同一Connection



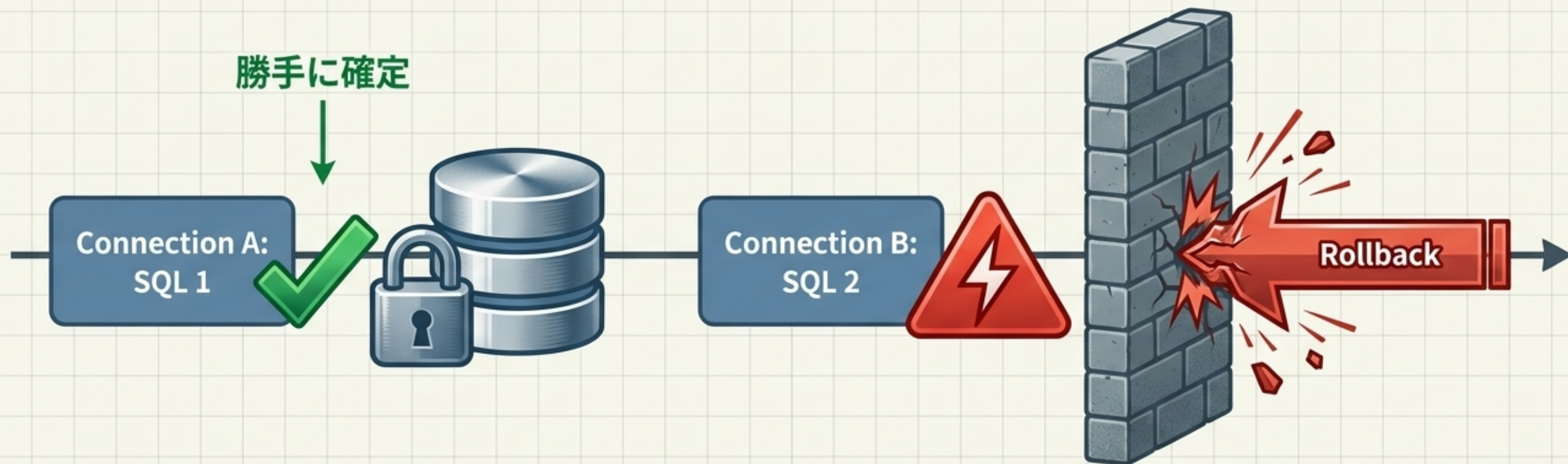
1つのConnection。
まとめて「全部成功 or 全部取り消し」が可能。

✗ 悪い例：別Connection



別々のConnection。トランザクションの単位が分断され、バラバラに確定されてしまう。

中途半端なデータの恐怖（別Connectionの末路）



- トランザクションの単位 = Connection
- 別Connectionで処理が走ると、前半の処理は別トランザクションとしてすでにDBに確定されてしまう。
- エラーが起きて rollback を呼んでも、別Connectionの確定済みデータには絶対に干渉できない。データ不整合（消えたお金、残ったゴミデータ）が確定する。

DAO トランザクション 3つの絶対掟



境界の法則

トランザクションの寿命と範囲は、1つの Connection オブジェクトに物理的に縛られる。途中で Connection を変えてはならない。



明示の法則

`setAutoCommit(false)` で自動確定を封印し、自らの手で `commit` または `rollback` の引導を渡す。



原状回復の法則

処理の成否に関わらず、必ず `finally` ブロックで `setAutoCommit(true)` に戻し、安全な状態でプールに返却する。